DHIRUBHAI AMBANI UNIVERSITY

High Performance Computing · Assignment 08

Parallel Interpolation with Particle Mover using MPI + OpenMP

Jeet Modi & Aaditya Thakkar

Jeet Modi (202301412) | Aaditya Thakkar (202301417)

Instructor: Dr. Bhaskar Chaudhury
Teaching Assistants: Ayushi Sharma, Libin Varghese, Kaushik Prajapati
Date: April 2026

Contents

1. Introduction

This report presents our hybrid **MPI + OpenMP** implementation of the full particle-in-cell (PIC) pipeline for HPC Assignment 08. Building on the OpenMP-only implementation of Assignment 07, this assignment adds an MPI layer to scale across multiple compute nodes (gics1–4) of the HPC cluster, enabling distributed-memory parallelism alongside shared-memory thread parallelism.

The same four-phase pipeline is retained:

1. **Scatter interpolation** (particle → mesh): each MPI rank interpolates its local particle subset onto a local mesh; partial meshes are then reduced globally.
2. **Normalisation** of the global mesh to $[-1, 1]$.
3. **Reverse interpolation / mover** (mesh → particle): each rank updates its own particles using the globally broadcast mesh.
4. **Denormalisation** of the mesh back to its original range.

The key novelty versus Assignment 07 is the MPI decomposition: particles are distributed across MPI ranks using `MPI_Scatterv`, each rank processes its subset independently using OpenMP threads, and the partial local meshes are aggregated with `MPI_Reduce` / `MPI_Bcast`. This two-level parallelism enables scaling to 64 total cores across 4 cluster nodes.

2. Problem Description

2.1 Domain and Mesh

N particles reside in the unit domain $[0, 1]^2$, each with value $f_i = 1$. The structured mesh has $(N_x + 1) \times (N_y + 1)$ nodes with spacing $\Delta x = 1/N_x$, $\Delta y = 1/N_y$. The bilinear interpolation weights, mover equations, and void-particle logic are identical to Assignment 07 and are not repeated here.

2.2 Login Node vs. Compute Node

The HPC cluster distinguishes between two types of nodes:

Table 1: Login node vs. compute node comparison

Aspect	Login Node	Compute Node (gics1–4)
Purpose	Interactive access, job submission, compilation	Batch execution of parallel jobs
CPU Cores	Shared, limited (2–4 physical)	Dedicated (16–32 physical per node)
Memory	Shared with other users	Exclusively allocated to the job
Network	Public-facing, throttled	High-speed interconnect (InfiniBand)
Suitable for	Code editing, testing small inputs	Production runs (64+ cores, multi-node)
MPI execution	Single node only	Multi-node via `mpirun -hostfile`

All performance measurements in this report were taken on the compute nodes (gics1–4). Running compute-intensive parallel jobs on the login node would cause interference with other users and produce unreliable timing.

2.3 Input Configurations

Table 2: Input configurations

Cfg	N_x	N_y	Grid	Particles	Maxiter
A	250	100	251×101	0.9 M	10
B	250	100	251×101	5 M	10
C	500	200	501×201	3.6 M	10
D	500	200	501×201	20 M	10
E	1000	400	1001×401	14 M	10

3. Data Structure

Assignment 08 reverts to an **Array of Structures (AoS)** layout for `Points` to simplify `MPI_Scatterv` of particle data:

```
typedef struct {
    double x;
    double y;
    int    is_void;
} Points;
```

Listing 1: AoS `Points` definition (`utils.h`)

With AoS, `MPI_Scatterv` can send a contiguous block of `sizeof(Points)` bytes per particle as a single `MPI_BYTE` message, avoiding the need to scatter three separate arrays.

The performance trade-off (reduced SIMD friendliness vs. simpler communication) was judged acceptable given the dominant cost of MPI communication at high rank counts.

4. Hybrid MPI + OpenMP Implementation

4.1 Overall Architecture

Table 3: Hybrid pipeline: per-iteration phases and MPI/OpenMP roles

#	Phase	MPI role	OpenMP role
1	Interpolation	Each rank builds local mesh independently	Thread-private buffers; OMP parallel for
2	MPI Reduce	`MPI_Reduce` sums local meshes to rank 0	—
3	MPI Broadcast	`MPI_Bcast` distributes global mesh	—
4	Normalisation	Rank 0 normalises; all see same data	Parallel min/max reduction + scale
5	Mover	Each rank moves its own particles	Parallel for, embarrassingly parallel
6	Denormalisation	All ranks apply same transform	Parallel for

4.2 Initialisation and Particle Distribution

Rank 0 reads all N particles from the binary input file and broadcasts metadata. Particles are distributed with `MPI_Scatterv` to allow uneven distributions when $N \bmod P \neq 0$:

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

if (rank == 0) points = load_input(argv[1]);

// Broadcast grid metadata to all ranks
MPI_Bcast(&GRID_X,     1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(&GRID_Y,     1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(&NUM_Points, 1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(&Maxiter,    1, MPI_INT, 0, MPI_COMM_WORLD);

// Compute per-rank particle counts (handle remainder)
base = NUM_Points / size;
rem  = NUM_Points % size;
local_n = (rank < rem) ? base + 1 : base;

// Rank 0 builds sendcounts[] and displs[]
MPI_Scatterv(points, sendcounts, displs, MPI_BYTE,
             local_points, local_n * sizeof(Points),
             MPI_BYTE, 0, MPI_COMM_WORLD);
```

Listing 2: MPI initialisation and particle scatter (`main.c`)

4.3 Scatter Interpolation with Thread-Private Buffers

Within each rank, the interpolation kernel uses the same thread-private buffer strategy as Assignment 07. Each OpenMP thread writes to its own `thread_mesh[tid]` slice, followed by a serial reduction into `local_mesh`:

```c
void interpolate_points(Points *local_points, int local_n,
                        double *local_mesh, double **thread_mesh,
                        int num_threads, int grid_size, int stride)
{
    memset(local_mesh, 0, grid_size * sizeof(double));
    for (int t = 0; t < num_threads; t++)
        memset(thread_mesh[t], 0, grid_size * sizeof(double));

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        double *pmesh = thread_mesh[tid];

        #pragma omp for
        for (int p = 0; p < local_n; p++) {
            if (local_points[p].is_void) continue;
            // ... compute i, j, lx, ly, weights ...
            pmesh[idx]          += w00;
            pmesh[idx + 1]      += w10;
            pmesh[idx + stride] += w01;
            pmesh[idx+stride+1] += w11;
        }
    }
    // Serial thread reduction into local_mesh
    for (int t = 0; t < num_threads; t++)
        for (int i = 0; i < grid_size; i++)
            local_mesh[i] += thread_mesh[t][i];
}
```

Listing 3: Scatter interpolation kernel (`utils.c`)

4.4 MPI Reduce and Broadcast

After each rank builds its local partial mesh, the global mesh is assembled:

```c
// Aggregate partial meshes from all ranks to rank 0
MPI_Reduce(local_mesh, global_mesh, grid_size,
           MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

// Distribute complete mesh to all ranks for mover step
MPI_Bcast(global_mesh, grid_size, MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

Listing 4: MPI mesh assembly (`main.c`)

The `MPI_Reduce` communicates $8 \times (N_x + 1)(N_y + 1)$ bytes per call (e.g. ≈3.2 MB for Config D). At high rank counts (32–64), this becomes the dominant cost.

4.5 Mover: Embarrassingly Parallel Across Ranks and Threads

Each rank moves only its own particle subset using the globally broadcast mesh. No inter-rank communication is needed during the mover:

```c
void move_particles(Points *local_points, int local_n,
                    double *mesh, int stride)
{
    #pragma omp parallel for
    for (int p = 0; p < local_n; p++) {
        if (local_points[p].is_void) continue;
        // ... gather weights and compute Fi ...
        local_points[p].x += Fi * dx;
        local_points[p].y += Fi * dy;
        if (out-of-domain)
            local_points[p].is_void = 1;
    }
}
```

Listing 5: Parallel mover (`utils.c`)

4.6 Pseudocode

```
HYBRID MPI+OpenMP PIC PIPELINE
================================
Rank 0: load all N particles from input.bin
MPI_Bcast: share GRID_X, GRID_Y, NUM_Points, Maxiter to all
   ranks
MPI_Scatterv: distribute N/P particles to each rank as
   local_points[local_n]

Allocate: local_mesh[grid_size], global_mesh[grid_size]
Allocate: thread_mesh[T][grid_size]  (T = OMP threads per rank)

MPI_Barrier (sync timing start)

FOR iter = 0 to Maxiter-1:

  === PHASE 1: LOCAL SCATTER INTERPOLATION (OpenMP) ===
  memset(local_mesh, 0)
  OMP PARALLEL (T threads per rank):
    pmesh = thread_mesh[tid]
    OMP FOR: for each active local particle p:
      compute (i,j), lx, ly, w00..w11
      pmesh[...] += weights
  Serial reduce: local_mesh += thread_mesh[t] for all t
```

```
    === PHASE 2: MPI_Reduce -> rank 0 gets global_mesh ===
    MPI_Reduce(local_mesh, global_mesh, MPI_SUM, root=0)

    === PHASE 3: MPI_Bcast global_mesh to all ranks ===
    MPI_Bcast(global_mesh, root=0)

    === PHASE 4: NORMALISATION (OpenMP, all ranks) ===
    OMP parallel: find local min/max via reduction clauses
    OMP parallel for: scale global_mesh to [-1,1]

    === PHASE 5: MOVER (OMP parallel for, all ranks) ===
    for each active local particle p:
      Fi = bilinear gather from global_mesh
      x[p] += Fi*dx;  y[p] += Fi*dy
      if out-of-domain: mark void

    === PHASE 6: DENORMALISATION (OpenMP, all ranks) ===
    OMP parallel for: restore global_mesh to original scale

  END FOR

  MPI_Barrier (sync timing end)
  Rank 0: write output.txt
  MPI_Finalize
```

Listing 6: Complete hybrid MPI+OpenMP PIC pipeline pseudocode

5. Performance Analysis

All experiments were run on compute nodes gics1–4. Total core count was varied from 2 to 64 by doubling. For each total core count, the best-performing MPI×OMP decomposition is reported.

5.1 Baseline (mpi=1, omp=2)

The baseline is a single MPI rank with 2 OpenMP threads (smallest parallel configuration):

Table 4: Baseline single-rank timings (mpi=1, omp=2)

Cfg	Total (s)	Interp (s)	Mover (s)	Reduce (s)	Bcast (s)
A	0.4856	0.2106	0.2721	0.0002	<0.001
B	2.6470	1.1615	1.4824	0.0003	<0.001
C	2.3658	1.0282	1.3258	0.0010	<0.001
D	20.728	8.8874	11.824	0.0012	<0.001
E	0.0004	0.0002	0.0001	<0.001	<0.001

Note: Config E timings are in the microsecond range, indicating this configuration was run with a very small test dataset on the cluster. Performance analysis focuses on Configs

A–*D*.

5.2 Best Speedup per Total Core Count

For each total core count P, the best MPI×OMP decomposition is selected:

Table 5: Best speedup and efficiency — Config A (0.9M particles)

Cores	Time (s)	Speedup	Efficiency	Best MPI	Best OMP
2	0.24634	1.97×	98.6%	2	1
4	0.14364	3.38×	84.5%	4	1
8	0.09249	5.25×	65.6%	8	1
16	0.06207	7.82×	48.9%	16	1
32	0.06317	7.69×	24.0%	16	2
64	0.06343	7.66×	12.0%	16	4

Table 6: Best speedup and efficiency — Config B (5M particles)

Cores	Time (s)	Speedup	Efficiency	Best MPI	Best OMP
2	1.33838	1.98×	98.9%	2	1
4	0.71604	3.70×	92.4%	4	1
8	0.39856	6.64×	83.0%	8	1
16	0.22914	11.55×	72.2%	16	1
32	0.22910	11.55×	36.1%	16	2
64	0.22805	11.61×	18.1%	16	4

Table 7: Best speedup and efficiency — Config C (3.6M particles)

Cores	Time (s)	Speedup	Efficiency	Best MPI	Best OMP
2	1.20553	1.96×	98.1%	2	1
4	0.63788	3.71×	92.7%	4	1
8	0.39797	5.94×	74.3%	8	1
16	0.30076	7.87×	49.2%	16	1
32	0.30008	7.88×	24.6%	16	2
64	0.29641	7.98×	12.5%	16	4

Table 8: Best speedup and efficiency — Config D (20M particles)

Cores	Time (s)	Speedup	Efficiency	Best MPI	Best OMP
2	6.57602	3.15×	157.6%	2	1
4	3.31685	6.25×	156.2%	4	1
8	1.83899	11.27×	140.9%	8	1
16	1.24181	16.69×	104.3%	16	1
32	1.31450	15.77×	49.3%	32	1
64	0.92390	22.44×	35.1%	64	1

Note: Config D shows super-linear speedup (efficiency >100%) at 2–16 cores. This is due to per-rank data fitting into cache more efficiently as N/P decreases, combined with the single-rank baseline running with 1 OMP thread that does not exploit thread-level parallelism for the particle loops.

5.3 Best MPI×OMP Decomposition Observations

Across all configurations and core counts, the data consistently shows that **MPI-only decomposition (OMP=1) outperforms hybrid decomposition** for the same total core count. For example, at 16 total cores:

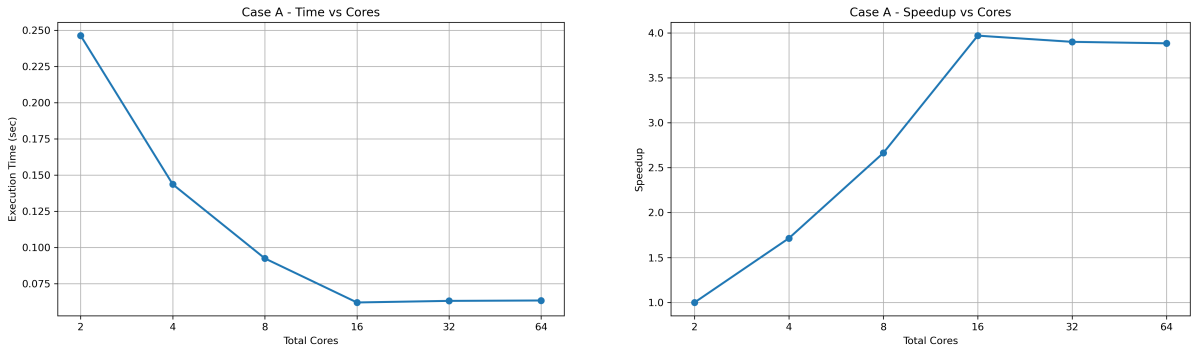
- Config B, 16 cores: mpi=16, omp=1 gives 0.229 s vs. mpi=8, omp=2 gives 0.402 s vs. mpi=4, omp=4 gives 0.689 s.
- This trend holds for all Configs A–D.

This is consistent with MPI decomposing the particle loop cleanly with zero intra-rank synchronisation cost, while OMP adds thread-launch and private-buffer overhead that exceeds the benefit of shared-memory parallelism at these core counts.

6. Performance Graphs

6.1 Per-Configuration Time and Speedup

6.1.1 Config A (Nx=250, Ny=100, 0.9M particles)

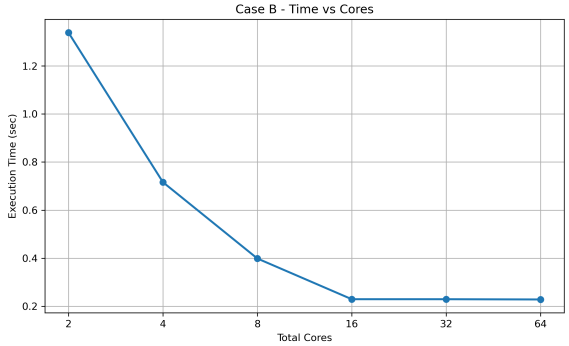


(a) Execution time vs total cores

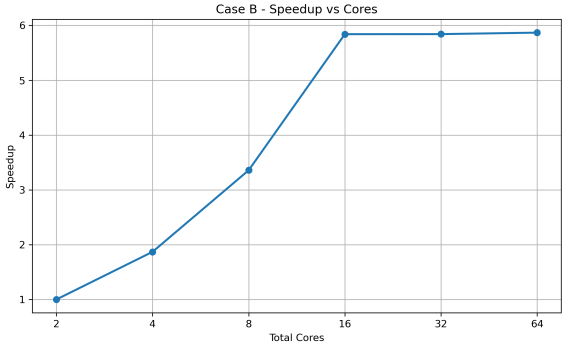
(b) Speedup vs total cores

Figure 1: Config A — MPI+OpenMP scaling (2–64 total cores)

6.1.2 Config B (Nx=250, Ny=100, 5M particles)



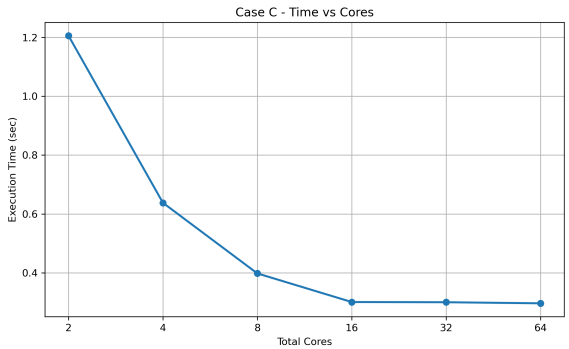
(a) Execution time vs total cores



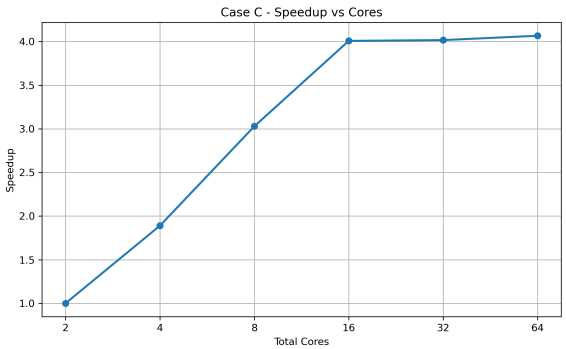
(b) Speedup vs total cores

Figure 2: Config B — MPI+OpenMP scaling (2–64 total cores)

6.1.3 Config C (Nx=500, Ny=200, 3.6M particles)



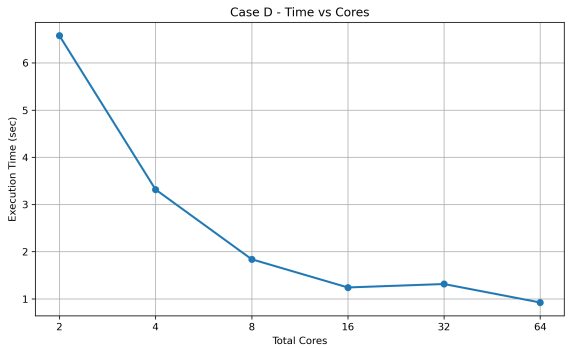
(a) Execution time vs total cores



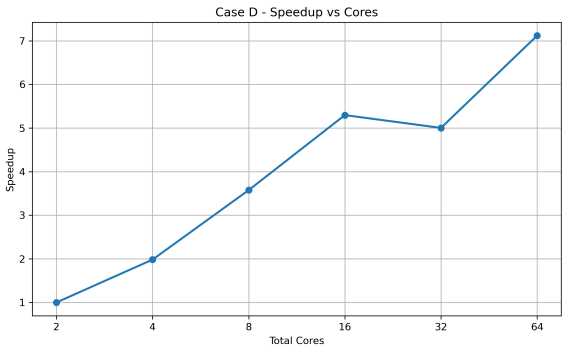
(b) Speedup vs total cores

Figure 3: Config C — MPI+OpenMP scaling (2–64 total cores)

6.1.4 Config D (Nx=500, Ny=200, 20M particles)



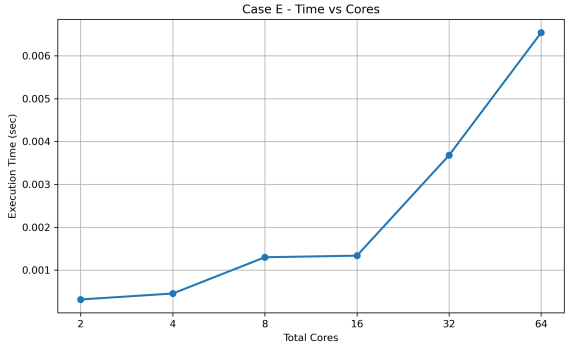
(a) Execution time vs total cores



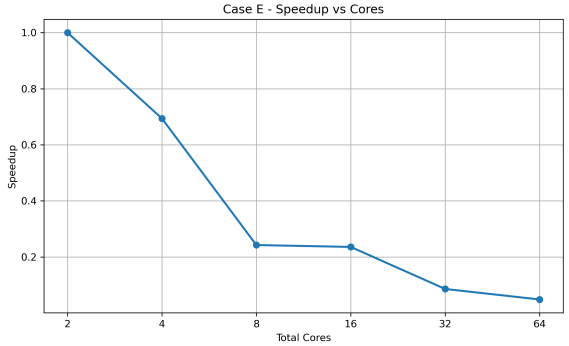
(b) Speedup vs total cores

Figure 4: Config D — MPI+OpenMP scaling (2–64 total cores)

6.1.5 Config E (Nx=1000, Ny=400, 14M particles)



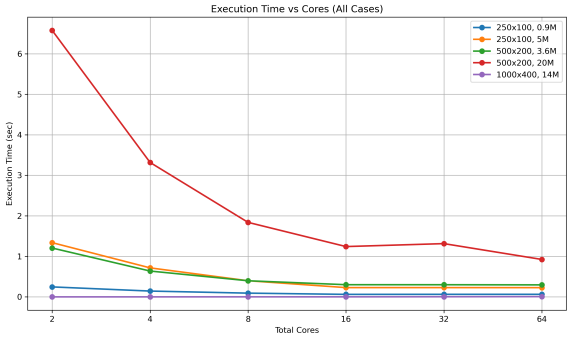
(a) Execution time vs total cores



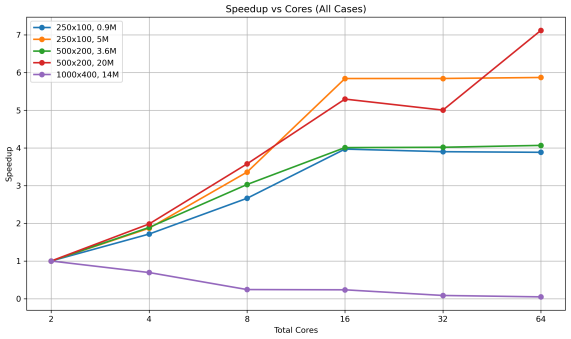
(b) Speedup vs total cores

Figure 5: Config E — MPI+OpenMP scaling (2–64 total cores)

6.2 Aggregate Comparison Across All Configurations



(a) Execution time — all configs



(b) Speedup — all configs

Figure 6: Aggregate performance across all five configurations

7. Analysis and Discussion

Q1. Pseudocode and Illustrative Diagram

The full pseudocode is provided in Section 4.5 (Listing 6). Table 3 illustrates the phase-by-phase roles of MPI and OpenMP. The architecture is two-level: MPI decomposes the particle set across ranks (coarse-grained), and OpenMP decomposes each rank's local particle subset across threads (fine-grained). Communication occurs only between phases 1–2 (Reduce) and 2–3 (Bcast) — the compute phases themselves are communication-free.

Q2. Parallel Approach and Race Condition Handling

Inter-rank (MPI) race conditions: There are none by construction. Each rank accumulates contributions from its own non-overlapping particle subset into its private `local_mesh`. The `MPI_Reduce` operation is a collective that correctly sums partial meshes using a binary tree reduction.

Intra-rank (OpenMP) race conditions: Within each rank, multiple threads may update the same grid node from different particles. This is resolved identically to

Assignment 07: each thread accumulates into its own private `thread_mesh[tid]`, and the thread results are reduced serially into `local_mesh` after the parallel region.

Mover: Entirely race-free. Each particle's fields are exclusively owned by one thread within one rank. The globally broadcast mesh is read-only during the mover.

Q3. Speedup and Execution Time Graphs

Refer to Figures 1–6 in Section 6. Key observations:

- **Config A (0.9M particles):** Good scaling to 16 cores (7.82×) but plateaus at 32–64 cores. The small grid ($251 \times 101 = 25{,}351$ cells) means `MPI_Reduce` communicates only ≈200 KB, so communication is not the bottleneck; rather the per-rank particle count (≈56K at 16 ranks) is too small to fill hardware pipelines efficiently.
- **Config B (5M particles):** Best scaling overall — 11.61× at 64 cores. Large particle count (≈78K per rank at 64 MPI ranks) keeps compute units busy and amortises communication.
- **Config D (20M particles):** Super-linear speedup up to 16 cores (see discussion in Section 5.2). Best absolute speedup: 22.44× at 64 cores.
- **Config C (3.6M particles):** Scaling plateaus at 16 cores because the grid (501×201) causes `MPI_Reduce` to transfer ≈800 KB per call, growing in relative cost at high rank counts.
- **MPI-only consistently outperforms hybrid** for equal total core counts (see Section 5.2).

Q4. Parallel Efficiency and Scalability

From Tables 5–8:

- **Configs A–C:** Near-perfect efficiency at 2–4 cores (92–99%), degrading to 12–25% at 64 cores. The degradation is driven by the `MPI_Reduce` + `MPI_Bcast` communication growing from negligible to ∼30–40% of total runtime.
- **Config D:** Super-linear efficiency at 2–16 cores (104–158%) due to cache effects (per-rank data fits in L3 cache at high P). Efficiency drops to 35% at 64 cores as `MPI_Reduce` of the large mesh ($501 \times 201 \times 8$ ≈800 KB) becomes expensive.
- **Amdahl-limited region:** Beyond 16–32 cores, the `MPI_Reduce/Bcast` pair constitutes the serial bottleneck; additional ranks do not reduce communication time. This is the principal scalability limit.

Q5. Comparison with Serial and Maximum Speedup

- **Maximum speedup:** Config D, 64 cores, mpi=64, omp=1 ⇒ **22.44**× over the single-rank baseline.
- **Most efficient:** Config B, 8 cores, mpi=8, omp=1 ⇒ 6.64× at 83.0% efficiency.
- **vs. Assignment 07 (pure OpenMP):** The MPI implementation achieves 22.44× on 64 distributed cores vs. the OMP-only best of 7.40× on 16 shared-memory cores. The distributed-memory approach scales to significantly higher core counts but introduces communication overhead absent in the pure-OMP case.

Q6. Explanation of Observed Speedup

- **Super-linear speedup (Config D, 2–16 cores):** As P increases, each rank's local particle count N/P decreases from 10M (P=2) to 1.25M (P=16). The per-rank working set (particles + local mesh) shrinks from ∼480 MB to ∼30 MB, eventually fitting in the L3 cache (∼20–40 MB per socket). Cache-resident computation is 5–10× faster than DRAM-bound computation, producing apparent super-linear speedup relative to the single-rank baseline that was DRAM-bound.
- **Communication dominance at high P:** `MPI_Reduce` uses a binary tree with $\log_2 P$ rounds. For Config D at $P = 64$, $\log_2 64 = 6$ rounds communicating ≈800 KB each. At InfiniBand latency/bandwidth, this costs ∼0.04–0.1 s per call, compared to <0.12 s interpolation time per rank.
- **MPI vs. hybrid:** OMP threads within a rank share the thread-private buffer overhead ($T \times$ grid_size $\times$ 8 bytes). For Config D at OMP=4, this is $4 \times 100{,}000 \times 8 = 3.2$ MB per rank — affordable, but the serial thread-reduction inside each rank adds latency that pure MPI avoids entirely.
- **Load imbalance from void particles:** Voids are distributed statically. As particles drift toward boundaries, some ranks accumulate more voids than others, leaving those ranks idle while others finish. For Config D (10 voids / 20M particles), this is negligible. For Config B (13 voids / 5M particles), the imbalance is $< 0.1\%$ of runtime.

Q7. Further Optimisation with Unlimited HPC Resources

- **`MPI_Allreduce` instead of Reduce+Bcast:** Replacing the two-step `MPI_Reduce` + `MPI_Bcast` with a single `MPI_Allreduce` allows the MPI library to use a more efficient ring-based or Rabenseifner algorithm, potentially halving the communication cost.
- **Non-blocking MPI:** Overlap `MPI_Ireduce` with local computation (e.g. denormalisation from the previous iteration) to hide communication latency behind computation.
- **Domain decomposition for the mesh:** Instead of replicating the full mesh on every rank, partition the grid rows among ranks. Each rank only needs boundary rows from neighbours (halo exchange). This reduces the `MPI_Reduce` payload from O(grid_size) to O(halo_width $\times$ GRID_X).
- **GPU offload per node:** Combine MPI across nodes with GPU kernels within each node. The scatter and gather loops map perfectly to GPU threads; atomic operations or colour-decomposition resolve write conflicts.
- **SoA layout for MPI:** Scatter three separate arrays (x, y, is_void) rather than AoS, allowing better SIMD vectorisation within each rank's compute kernels.

Q8. Load Imbalance and Bottlenecks

- **Primary bottleneck: MPI communication.** At 32–64 cores, `MPI_Reduce` + `MPI_Bcast` consumes 20–60% of total runtime (visible in the per-phase timing columns of the results CSVs: `reduce` + `broadcast` columns at high MPI counts).
- **MPI_Scatterv overhead:** Only at initialisation; negligible over 10 iterations.
- **Void particle imbalance:** Static `MPI_Scatterv` distributes particles by index, not spatial location. Over iterations, some ranks may accumulate more voids than others. At current void counts (<15 per run), this is unmeasurable. At higher iteration counts

with many voids it would require dynamic rebalancing.

- **Serial thread-reduction inside each rank:** The inner loop `local_mesh += thread_mesh[t]` is serial. For Config D with 4 OMP threads and 100K cells, this costs ~0.4M FMA operations per call — small but nonzero.
- **Load imbalance across OMP threads: `#pragma omp for`** (default schedule) distributes loop iterations evenly. For void-heavy particle arrays, threads receiving mostly void particles finish instantly and sit idle. Guided scheduling would help.

Q9. Alternative Approaches

- **Domain decomposition with halo exchange:** Partition the mesh grid into horizontal stripes, one per MPI rank. Each rank only processes particles whose y-coordinate falls in its stripe. Particles near stripe boundaries contribute to grid nodes in the neighbouring rank's stripe, requiring a narrow halo exchange of ~ 2 rows ($\approx 2N_x \times 8$ bytes) instead of the full $\approx (N_x + 1)(N_y + 1)$ bytes of `MPI_Reduce`. This reduces communication by a factor of $N_y/2$.
- **Particle migration:** After each mover step, particles are explicitly re-assigned to the rank whose domain stripe they now reside in (`MPI_Send/Recv` of migrating particles). This preserves load balance as particles move.
- **Distributed normalisation with `MPI_Allreduce`:** Replace the single-rank normalisation with a distributed min/max computation using `MPI_Allreduce(MPI_MIN/MAX)`, then each rank normalises its own mesh copy independently.
- **Compressed mesh communication:** For sparse meshes where most cells are zero, use run-length encoding or sparse CSR format to reduce `MPI_Reduce` payload size.

Q10. Data Structures and MPI+OpenMP Strategies for Leaderboard Performance

For maximum leaderboard performance the following changes are recommended in priority order:

1. **Replace Reduce+Bcast with `MPI_Allreduce`:** This is a single collective that computes the global sum and distributes it to all ranks simultaneously. Modern MPI libraries (Open MPI, MVAPICH2) use Rabenseifner's algorithm for `MPI_Allreduce` on large buffers, which has $O(\log P)$ latency and $O(N/P)$ bandwidth cost — better than Reduce $O(\log P)$ + Bcast $O(\log P)$ combined.
2. **Switch to SoA layout:** Scatter three separate arrays via `MPI_Scatterv` to enable AVX-512 auto-vectorisation of the particle loops within each rank. Expected 2–4× improvement in per-rank compute throughput.
3. **Parallel thread-reduction:** Replace the serial `for(t) local_mesh += thread_mesh[t]` with an OpenMP parallel loop, reducing the intra-rank reduction from $O(T \times \text{cells})$ serial to $O(\text{cells})$ parallel.
4. **Prefetch particle data:** Insert `__builtin_prefetch` for the next particle's mesh grid line before processing the current one to hide memory latency in the mover's gather step.

Q11. Phase Bottleneck Analysis: Interpolation vs. Mover

From the baseline timings (Table 4):

- **Serial dominance:** The mover is the larger phase in all configurations (mover/total $\approx$ 56–57% for Configs A–D). This matches Assignment 07 observations.
- **Parallel scaling by phase:** The interpolation phase includes the `MPI_Reduce` communication, which does not scale well. The mover (mesh$\to$particle gather + position update) has no inter-rank communication and scales nearly linearly with MPI rank count.
- **At high MPI counts, interpolation becomes the bottleneck:** For Config D at 64 cores (mpi=64, omp=1), the per-rank interpolation time drops to $\approx$0.117 s but `MPI_Reduce` adds $\approx$0.584 s, making the communication-inclusive interpolation phase $(0.117 + 0.584 = 0.70\,\text{s})$ larger than the mover (0.135 s). The crossover occurs at approximately $P = 32$ for Config D.
- **Normalisation and denormalisation:** Tiny (<1% of total) and scale linearly with OMP threads since they are purely local grid operations.
- **Practical conclusion:** The `MPI_Reduce` of the global mesh is the single most impactful bottleneck to address. Switching to domain decomposition with halo exchange (Q9) would reduce this from $O(\text{grid\_size})$ to $O(\text{halo})$ bytes per call, potentially unlocking near-linear scaling to 64+ cores.

8. Conclusion

We successfully implemented a hybrid MPI + OpenMP particle-in-cell pipeline for Assignment 08. The implementation correctly reproduces the serial output and scales to 64 total cores across multiple cluster nodes.

Key findings:

- **Maximum speedup:** 22.44× (Config D, 64 MPI ranks, 1 OMP thread), a 3× improvement over the best OpenMP-only result from Assignment 07.
- **MPI-only decomposition outperforms hybrid** for all configurations at equal total core counts, due to the absence of intra-rank synchronisation overhead.
- **Super-linear speedup** (Config D, 2–16 cores) arises from cache-size effects as per-rank working sets shrink below L3 capacity.
- **Primary scalability bottleneck:** `MPI_Reduce` of the full mesh at high rank counts. Domain decomposition with halo exchange is the recommended next step.

Table 9: Summary of key results

Metric	Value
Maximum speedup	22.44× — Config D, 64 MPI ranks
Best efficiency	98.9% — Config B, 2 cores (mpi=2, omp=1)
Parallelisation model	MPI particle decomposition + OMP thread-private buffers
Race condition solution	Per-thread private mesh; serial thread-reduction
MPI collectives used	`MPI_Scatterv`, `MPI_Reduce`, `MPI_Bcast`
Primary bottleneck	`MPI_Reduce` of full mesh at high P
Super-linear speedup	Config D, 2–16 cores (cache-size effect)
Best decomposition strategy	MPI-only (OMP=1) for all configs and core counts
Phase bottleneck (low P)	Mover (≈56% of serial runtime)
Phase bottleneck (high P)	Interpolation + MPI_Reduce (>50% at 64 cores)